

Polimorfismo

João Eduardo Montandon
DCC024

```
int f(char c, char b) {  
    return a == b;  
}
```

```
fun f(a, b) = (a = b);
```

Qual a diferença entre as duas? 🤔

Enquanto a primeira lida apenas com caracteres, a segunda lida com qualquer tipo comparável!

Uma função ou operador é polimórfico quando ele assume pelo menos dois tipos possíveis

Tipos De Polimorfismo

1. Sobrecarga
 2. Coerção
 3. Paramétrico
 4. Subtipos
-

Sobrecarga

Acontece quando uma função ou operador possuem pelo menos duas definições envolvendo tipos distintos

```
int square(int n) {
    cout << "int square(int)" << endl;
    return n * n;
}

double square(double n) {
    cout << "double square(double)" << endl;
    return n * n;
}

int main() {
    cout << square(2) << endl;
    cout << square(2.0) << endl;
    return 0;
}
```

Como linguagens implementam sobrecarga? 🤖

```
g++ -S overload.cpp
```

Muitas linguagens suportam sobrecarga com operadores

```
class Fracao:
    def __init__(self, n, d):
        self.n = n
        self.d = d
    def __mul__(self, outraFracao):
        n = self.n * outraFracao.n
        d = self.d * outraFracao.d
        return Fracao(n, d)
    def __str__(self):
        return f"{self.n}/{self.d}"

f1, f2 = Fracao(1, 2), Fracao(3, 1)
f3 = f1 * f2
print(f3)
```

Coerção

Conversão implícita de um tipo para outro

```
double x;  
x = 2;  
System.out.println(x); // 2.0
```



```
double x;  
x = (double) 2;  
System.out.println(x); // 2.0
```

Mas isso é polimorfismo? 😬

Coerção De Parâmetros

Sobrecarga utiliza os tipos para escolher a definição a ser utilizada



A coerção usa a definição para escolher o tipo a ser convertido

```
/*  
int square(int n) {  
    cout << "int square(int)" << endl;  
    return n * n;  
}  
*/  
  
double square(double n) {  
    cout << "double square(double)" << endl;  
    return n * n;  
}  
  
int main() {  
    cout << square(2) << endl;  
    cout << square(2.0) << endl;  
    cout << square('a') << endl;  
    return 0;  
}
```

E se criarmos uma função `float square(float n)`? 😬

```
void f(int x, char y) {
    cout << "f(int, char)" << endl;
}

void f(char x, int y) {
    cout << "f(char, int)" << endl;
}

void main() {
    f('a', 'b');
}
```

O que vai ser impresso? 😬

Paramétrico

Uma função possui polimorfismo paramétrico se possuir pelo menos um tipo variável.

O que é tipo variável? 😬

```
template <class X> X maximo(X a, X b) {
    return a > b ? a : b;
}

void g(int a, int b, int c, char d) {
    cout << maximo(a, b) << endl;
    cout << maximo(c, d) << endl;
}
```

```
int main() {
    g(10, 20, 'a', 'z');
    return 0;
}
```

Como linguagens implementam polimorfismo paramétrico? 😬

```
g++ -S parametric.cpp
```

Soa [familiar](#)? 😬

Outro exemplo: ML

```
- fun equals(x, y) = (x = y);  
Warning: calling polyEqual  
val equals = fn : 'a * 'a -> boo
```

Polimorfismo Por Subtipagem

Uma função ou operador exibe polimorfismo de subtipo se pelo menos um parâmetro possuir subtipos

- Se um subtipo é um subconjunto ([Aula Tipos](#))
 - Então, toda função que manipula um tipo, também é capaz de manipular um subtipo
-

```
public class Subtipos {  
    public static void imprime(Object o) {  
        System.out.println(o);  
    }  
    public static void main(String[] args) {  
        print(new String("dcc024"));  
        print(new Integer(42));  
        print(new Character('a'));  
    }  
}
```

Considerações Finais

- *Polimorfismo ad hoc* ➡ quando envolve um número finito de tipos
 - Sobrecarga
 - Coerção
- *Polimorfismo universal* ➡ quando envolve um número infinito de tipos
 - Polimorfismo paramétrico

- Polimorfismo por subtipos

-
- Existem várias outras formas de polimorfismo
 - Variáveis polimórficas, classes, pacotes, linguagens